Scrape Company Career Page for Job Listings

Project Objective

To develop a Python script that scrapes job postings from a target company's public careers page and organizes the extracted information into a structured Excel file.

Task Description

The goal is to programmatically collect key details about open job positions from a single company's website. This data can be used for market analysis, tracking opportunities, or building a job aggregation database. The script should be robust enough to handle missing information gracefully and save the final data in a clean, user-friendly format.

For this task, you can choose the careers page of any major company. Examples include the career pages of Google, Microsoft, Amazon, or a popular job board.

Scope of Work & Methodology

1. **Environment Setup:**
    - Set up a local Python environment.
    - Install the necessary libraries:
        - requests (for making HTTP requests to the website)
        - BeautifulSoup4 or parsel (for parsing HTML content)
        - pandas (for structuring data)
        - openpyxl (to enable pandas to write .xlsx files)
2. **Website Inspection:**
    - Choose a target company's careers page.
    - Using your web browser's Developer Tools (F12), inspect the HTML structure of the page.
    - Identify the specific HTML tags, classes, and IDs that contain the data points for each job listing (e.g., the div that holds a single job card, the h2 for the title, the span for the location).
3. **Scraping Logic Development:**
    - Write a Python script to send an HTTP GET request to the careers page URL.
    - Parse the returned HTML content using BeautifulSoup.

SuperAI
Smart. Success. Superior.

- Identify the main container that holds all the job listings.
- Loop through each individual job listing element within the main container.
- For each listing, extract the required data fields (see section 5). Implement error handling (e.g., using try-except blocks or conditional checks) to manage cases where a field like 'Salary' or 'Experience' is not specified.
- If the website has multiple pages of listings (pagination), your script should be able to identify the "Next" page link and continue scraping until all pages have been processed.

4. **Data Structuring and Export:**
   - Store the scraped data for each job as a dictionary. Append each dictionary to a list.
   - Convert the list of dictionaries into a pandas DataFrame.
   - Export the DataFrame to an Excel file (.xlsx). Ensure the file is well-formatted and does not include the pandas index column.

Data Fields to be Scraped (Excel Columns)

Your final Excel file must contain the following columns:

Column Name	Description
`JobTitle`	The title of the job position.
`Location`	The city, state, or country where the job is located.
`ExperienceRequired`	The years or level of experience required (e.g., "5+ years", "Senior").
`SkillsRequired`	A list or comma-separated string of required skills.
`Salary`	The salary or salary range, if specified. Leave blank if not available.
`JobURL`	The direct URL to the detailed job description page.
`JobDescriptionSummary`	A brief summary or the first few lines of the job description.

Supe AI
Smart? Success? Superior?

Required Technical Stack

- **Language:** Python 3.x
- **Core Libraries:** requests, BeautifulSoup4, pandas, openpyxl

Deliverables

1. **Excel File:** A single .xlsx file named [CompanyName]_Jobs.xlsx (e.g., Google_Jobs.xlsx) containing the scraped data with the columns specified in Section 5.
2. **Python Script:** The complete, commented Python script (.py) used to perform the scraping.

Acceptance Criteria

- The Python script runs without errors.
- The Excel file is correctly formatted and contains all the required columns.
- The data accurately reflects the information on the live careers page.
- The script gracefully handles missing data points (e.g., salary, experience) by leaving the corresponding cells in the Excel file blank.
- The script can handle pagination if present on the target site.

Smart. Success. Superior.